本章路線

這章要回答的核心問題是：一個以 UNIX 傳統為基礎的核心，如何組合程序、資源與模組介面？

前置知識。 process/thread、virtual memory、檔案系統與 module。

- 區分歷史 Linux 設計與現行系統
- 追蹤 fork/exec、module load、page cache 路徑
- 分析書中 CFS 與其前身、page reclaim 及 file system 的版本限制

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel、system libraries與distribution不是同一層

本章固定Linux v3.2核心，依OSC9e的歷史案例讀機制。核心提供task、virtual memory、VFS、IPC、network與drivers；GNU工具、shell、libc與套件管理器大多在user space。distribution把核心、工具、設定與安裝/更新流程整合成產品，不能從distribution名稱反推所有kernel設定。

固定兩台教學機均用v3.2、同一x86 ISA，執行的ELF檔案含`PT_INTERP=/lib/ld-linux.so.2`。A有此interpreter，B沒有。兩者都找得到可執行檔，但B會在取得interpreter時失敗；這不表示「同一kernel version保證所有user binaries可執行」。靜態連結、動態連結、ISA與ABI、library symbol版本是不同相容性條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel、system libraries與distribution不是同一層 (續)

元件	執行權限/責任	失敗的可觀察結果
kernel image及其CONFIG	核心模式；提供syscall與裝置支援	缺driver可使裝置無法使用
ELF interpreter/libc	通常user mode；載入共享libraries與API包裝	interpreter缺少可使exec失敗；symbol缺少可使loader失敗
shell/utilities	user processes；解析命令與使用服務	某命令不存在不代表kernel沒有fork/read
distribution policy	套件與預設設定的整合	不同設定可產生不同權限/啟動行為

Linux從1991年的初期核心逐步擴充至SMP與多種架構；可載入模組、細粒度鎖與標準介面回應了硬體多樣性、多人多工及相容性需求。歷史脈絡解釋設計取捨，具體行為仍由所指定原始碼與設定支持。

Modular monolithic的邊界在哪裡

Linux把scheduler、memory、VFS及多數drivers放在同一kernel protection domain。模組可獨立載入，但不是各自有硬體隔離的user servers；合法function pointer呼叫即可進入driver，不因跨module就必須切換address space。

固定單一同步buffered read：user以syscall進核心，VFS呼叫filesystem，再呼叫block/driver。只跨user/kernel邊界會發生mode transition；核心內函式呼叫不等於mode switch。若等待I/O時scheduler改跑另一task，才另有context/address-space切換，不能把所有read都固定計成同一個切換數。

隔離與維護性的取捨由此產生。一個driver用錯pointer可破壞同kernel內的資料；module ABI、型別檢查或簽章驗證不會把它變成另一個故障隔離domain。反過來，將檔案服務移到user process可利用page-table protection，但必須設計IPC、資料傳輸與server失敗後的協定。第2章用固定配置比較兩種read路徑，本章則把Linux核心內部的物件接起來。

Linux的UNIX/POSIX介面讓程式操作file descriptors、processes與signals；API相容是可觀察語意的契約，與核心內部是否採某一份資料結構不同。相同read API可以由磁碟、pipe或socket提供，各自的blocking、seek與EOF條件仍不同。

模組載入是一個資源交易

以下是自行設計的「最後發布」協定，並非 Linux 3.2 對所有註冊入口的保證。固定 driver module M 需要 exported symbols A/B並註冊device D。載入依序進行格式/版本檢查、配置與relocation、解析允許的symbols、執行init、公布註冊結果。若只有A可用而B未解析，不能公布一個已可呼叫D的半成品；若init取得buffer後註冊D失敗，須回收buffer與先前資源。

階段	M的可見性	取得的資源	失敗出口
resolve前	不可供一般呼叫	載入區域	驗證失敗即拒絕
resolve A/B	尚未LIVE	relocation後的code/data	未解析symbol時釋放載入資源
init	尚在建立服務	queue、buffer；尚未發布 device registration	反向撤銷已成功取得者
LIVE	新呼叫可取得合法引用	已註冊服務	每個呼叫遵守生命週期

Linux 3.2 的部分註冊入口可在 init 返回前對外可見；驅動必須在註冊前完成外界可見狀態的初始化，並在 init 失敗時撤銷入口、同步並行呼叫及回收資源。上表的「init」在本模型中僅指私下準備、尚未對外發布的階段。

模組載入是一個資源交易 (續)

M與kernel同權限。signature enforcement依版本與CONFIG決定，不能當成Linux3.2所有模組必備的前提；它與runtime引用安全解不同問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何reference count等於0仍需停止新呼叫

固定安全卸載模型，每個open handle及每個queued request各持有一個M引用，初始3個handles+2個requests=5。狀態變化如下，表中的引用政策是本driver的明定規則。

階段	可新增引用	handles	requests	refs	code可釋放？
LIVE	可以	3	2	5	否
GOING/撤銷新入口	不可以	3	2	5	否
關閉3個handles	不可以	0	2	2	否
兩筆I/O完成或合法取消	不可以	0	0	0	尚須確認所有callback已離開
callback/worker皆靜止	不可以	0	0	0	才可釋放

若先看到refs=0，再撤銷入口，另一CPU可能在兩步間取得引用並開始執行即將被釋放的code。須由module狀態/註冊協定阻止新進入，再完成既有引用與工作。使用RCU的callback要等其相應grace period；使用workqueue、timer或DMA者要完成各自的同步取消，不能宣稱所有module等「一次RCU」就安全。

fork 複製哪一層：task、mm、頁表與資料頁

Linux 3.2 的 `copy_process()` 呼叫 `dup_task_struct()`，先配置 child 的 task 與 kernel stack，再初始化 PID、credentials、scheduler state 與其他 execution context。一般 `fork()` 未設定 `CLONE_VM`，`copy_mm()` 走 `dup_mm()`：child 有新的 `mm_struct` 與頁表。`copy_page_range()` 可以讓 private user pages 先共用 physical frame 並寫入保護；某些 VMA 的頁表複製可以延後到 fault 建立，並非所有頁表項都必須立即複製。write fault 需要隔離時才複製；若匿名頁已可安全獨佔，核心可能重用原頁並恢復可寫。整條路徑沒有「等到第一次寫 task metadata 才複製 `task_struct`」的步驟。

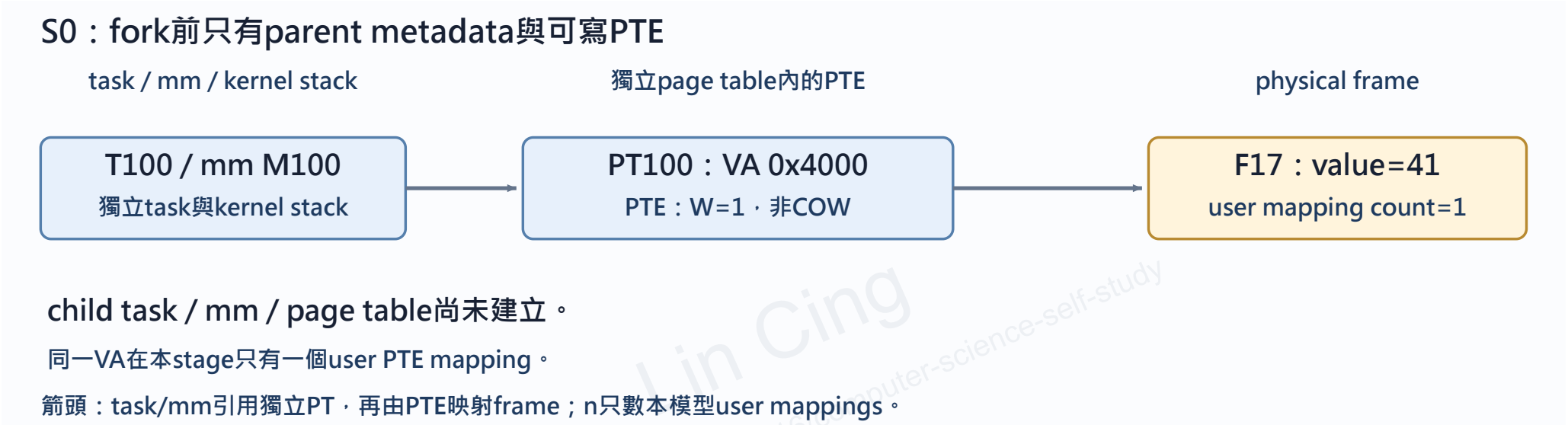
層次	fork 後父子關係	首次 private write 的作用
task_struct／kernel stack	不同物件，各有 PID、排程狀態與核心 stack	不以資料頁 COW 延後建立
mm_struct／頁表	一般 fork 為不同物件	child 更新自己的 PTE
適用且已駐留的匿名私有頁	可共用同一 frame；兩個 PTE 暫時禁止寫入	write-protection fault 需要隔離時才分離複製；安全獨佔時可重用
open file description	對應 descriptor 可以仍指向同一物件	offset 可共享，與 user page COW 不同

fork 複製哪一層：task、mm、頁表與資料頁 （續）



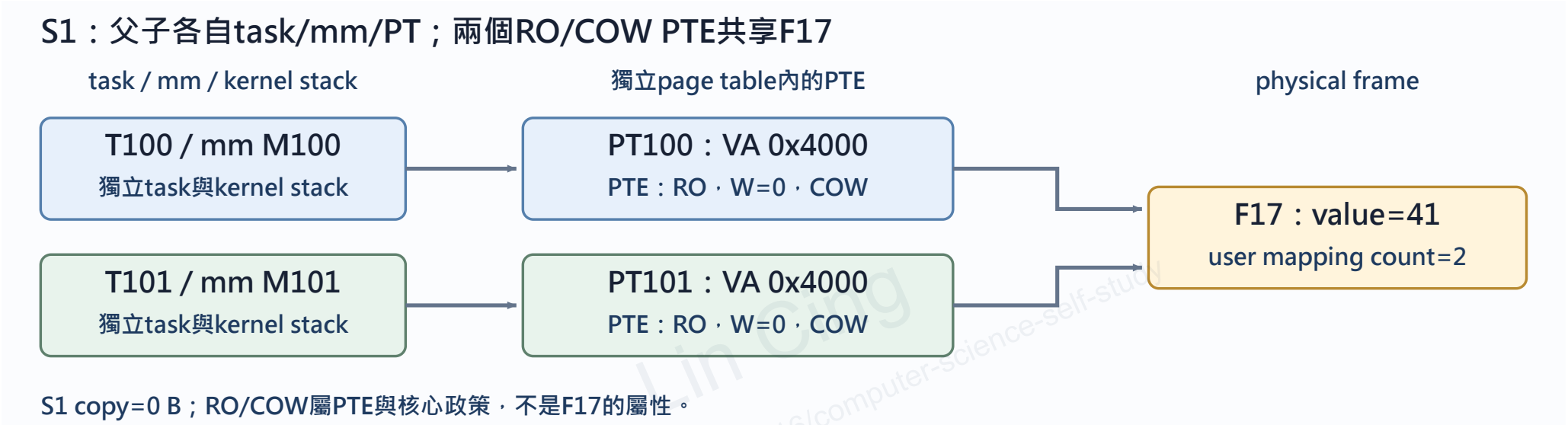
`clone()` 是資源分享的參數化入口。`CLONE_VM` 讓兩個 task 指向同一 mm；`CLONE_FILES` 共享 descriptor table；`CLONE_FS` 共享 filesystem context；`CLONE_SIGHAND` 共享 signal-handler table。這些 flags 可形成 thread-like task，但不能拿來證明一般 fork 共用 task metadata。

S0：fork 前只有 parent task



固定 parent 為 T100/M100/PT100，VA 0x4000 映射 frame17，x=41、W=1、mapping count=1；child task 尚未存在。此狀態定義後續比較的基準，不能先假設 COW 或 child metadata 已存在。

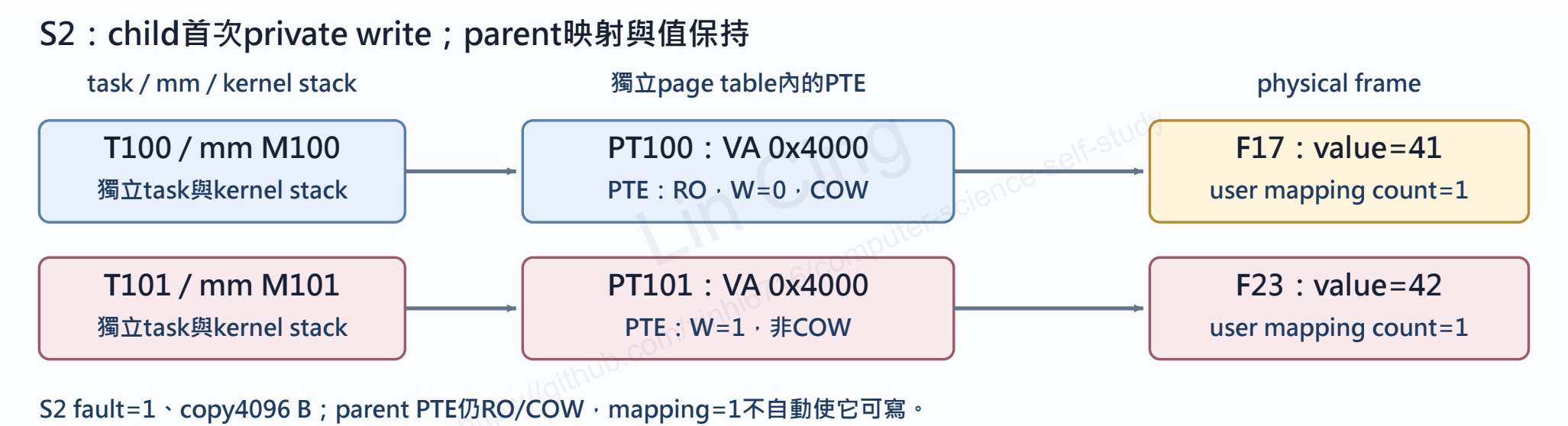
S1：fork 後 metadata 已分離，frame 仍共享



child 為 T101/M101/PT101，PID、task state、kernel stack、mm 與 page table 都是自己的物件。兩張 PTE 暫時都映射 frame17、禁止寫入並標記 COW；F17 內容仍為 x=41，user PTE mapping count=2，資料複製 bytes=0。

S2：child 第一次寫入才複製 private page

假設只有一個 4 KiB anonymous private page，VMA 邏輯上可寫；x 位於 VA 0x4000、頁內 offset 0，初值 41。無 huge page、額外 mapping、DMA 或同時寫入。表中 COW 是核心政策標籤，不宣稱 x86 PTE 有名為 COW 的硬體位元；計數只算此模型的 user PTE mappings，不等於 Linux page 的全部 kernel references。



S2：child 第一次寫入才複製 private page （續）

狀態	parent task／mm／PTE	child task／mm／PTE	frame 內容與 mapping 數
S0 fork 前	T100／M100／PT100: frame17,W=1	不存在	F17:x=41，n=1
S1 fork 後	T100／M100／PT100: F17,W=0,COW	T101／M101／PT101: F17,W=0,COW	F17:x=41，n=2；資料複製0 bytes
S2 child 寫 x=42	T100／M100／PT100: F17,W=0,COW	T101／M101／PT101: F23,W=1	F17:x=41,n=1；F23:x=42,n=1；fault1次、複製4096 bytes

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

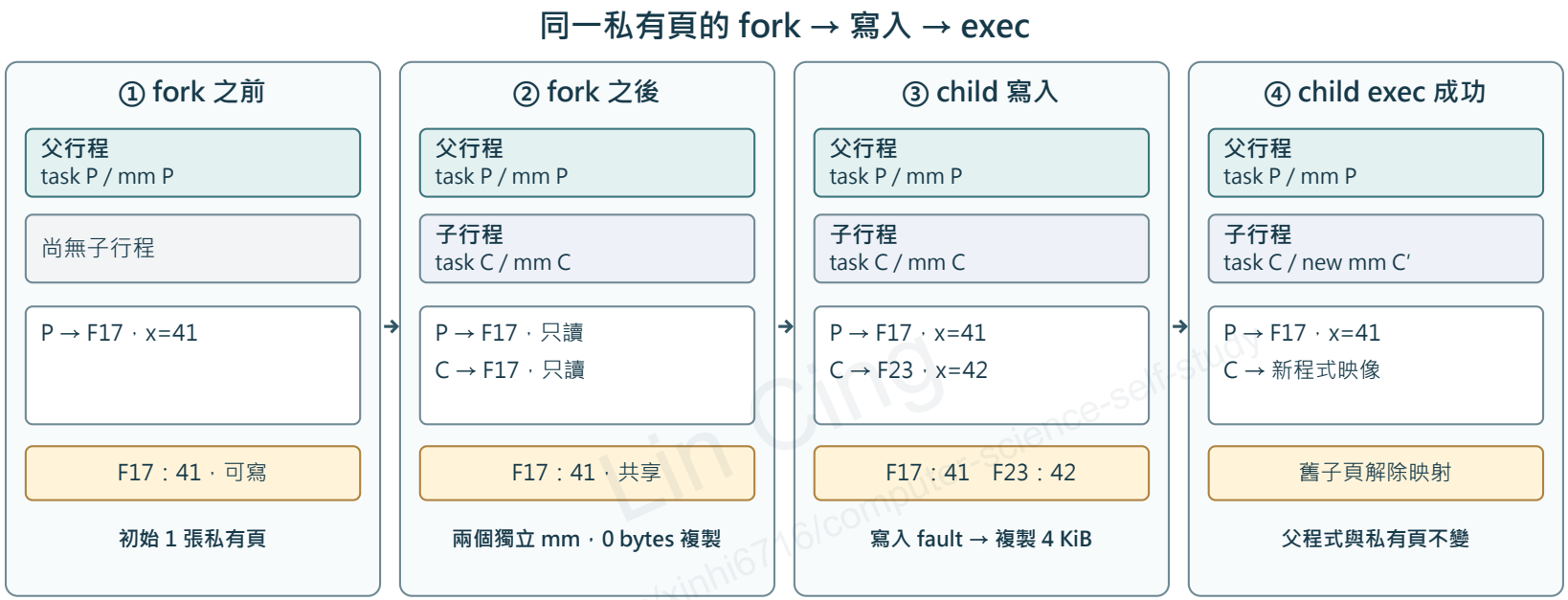
錯誤模型的可觀察反例與 clone 對照

若誤把 child task 設為 parent task 的同一物件，再把 child PID 改為101，parent PID 也變成101；兩個程序不再有獨立身分。正確模型中 child state 由 ready 改 blocked，parent state 仍是 ready。資料頁共享不能推成 task 共享。在本節single-thread child模型中，`exec()`成功後保持process PID而換入program image；一般在破壞舊image前失敗可回傳錯誤，但Linuxv3.2 `flush_old_exec()`之後已跨過point of no return，後續部分錯誤會送signal終止，不能無條件保證還原舊image。

`CLONE_VM` 是另一個明定的共享選項：兩個不同 task 可指向同一 mm／頁表。若初始 F17 可寫，child 寫42，parent 讀到42，而且不因本次寫發生 private-page COW fault。這個結果與上表一般 fork 的 parent=41、child=42 不同。

三態表與 CLONE_VM 對照只驗證本節固定模型的可觀察差異，不代表實際 Linux 核心測試。真核心行為仍須以對應版本的 source、fault trace 與 frame/PTE counters 驗證。

fork、寫入與 exec：先分清物件身分



固定模型：private anonymous page；無併發寫入、額外映射與 DMA

本例只有一個可寫的 private anonymous page。fork 後父子各有 task 與 mm，暫時共享 F17；當兩者仍共享時，child 寫入觸發 fault，複製後才寫 F23。成功 exec 替換 child 的程式映像，parent 保持自己的映像。

fork、寫入與 exec：先分清物件身分 （續）

一般情況下，fault **必要時**才複製；若頁已可安全獨佔，核心可能重用。`MAP_SHARED` 不採本例的私有寫入語意；已開啟的 file descriptor 也可能繼續引用同一 open file description，與記憶體頁的 COW 是不同關係。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

clone flags不是可任意排列的布林組合

建立模式	task/TID	TGID	mm	fd table	file descriptions
一般fork	新	新group	新mm，private pages可COW	複製table	原有fd仍引用同一descriptions
只加CLONE_VM	新	仍可新group	共享mm	可複製table	引用可共享
thread-like，含VM/SIGHAND/THREAD	新	共享thread group	共享mm	是否共享另看CLONE_FILES	依table entries引用

Linuxv3.2拒絕CLONE_THREAD卻沒有CLONE_SIGHAND，也拒絕CLONE_SIGHAND卻沒有CLONE_VM。因此一般pthread模型除了資源sharing flags，還要CLONE_THREAD等thread-group語意；只列VM/FS/FILES/SIGHAND四個flags就說成完全等同pthread，會漏掉TGID與signal/thread-group生命週期。

例如parent TID100/TGID100，普通fork child可為TID101/TGID101；同group thread可為TID102/TGID100。兩者都有獨立task與kernel stack，不能因CLONE_VM共享mm就把兩個TIDs當成同一個task物件。

CFS把實際CPU時間換成virtual runtime

固定Linux3.2的fair scheduling class。runnable entities依virtual runtime排序，權重較高者在相同實際時間內增加較少的virtual runtime，因此可獲較多CPU。以nice0的基準weight1024：

$$\Delta v = \Delta t \frac{1024}{w}.$$

兩個持續runnable tasks A(nice0,w1024)、B(nice5,w335)，固定目標period6ms、無其他負載。理想比例為A=1024/1359、B=335/1359；對應約4.521ms與1.479ms。兩者的virtual increments均約4.521ms，因B以較小weight換算。這是權重公平性的算式，不是宣稱每一個實際period都精確切成這兩段；tick、wakeup、min granularity及其他classes會改變逐段時序。

CFS的rbtree保存的是fair class的排序，running entity的入/出tree由調度路徑處理；不能把「最左者」簡化成每一時刻所有tasks都在同一tree。RT FIFO/RR有自己的priority規則，可優先於fair class；本章不把它套進同一個vruntime比較。第6章的HRTICK與periodic-tick兩份固定Gantt提供實際選取的中間值。

權重如何決定 CPU 份額與 vruntime

CFS 權重：從實際時間換算虛擬時間

單一 CPU，A、B 持續 runnable；理想共享期間 6 ms

總權重 1024 + 335 = 1359　　$\Delta v = \Delta t \times 1024 / \text{權重}$

權重比例	實際 CPU 時間	虛擬時間增量
A：nice 0，權重 1024 1024 / 1359 ≈ 75.35%	6 × 1024 / 1359 ≈ 4.521 ms	6 × 1024 / 1359 ≈ 4.521 ms
B：nice 5，權重 335 335 / 1359 ≈ 24.65%	6 × 335 / 1359 ≈ 1.479 ms	6 × 1024 / 1359 ≈ 4.521 ms

這是理想份額的算術模型，並非每個排程週期的實際執行順序。

Linux 3.2 的理想模型中，兩者持續 runnable 且只考慮一顆 CPU。較小權重 B 的實際時間較少，以未四捨五入的時間經權重換算後的虛擬時間增量與 A 相同；實際排程還受 tick、喚醒與其他 class 影響。

local IRQ、preemption與SMP互斥各保護什麼

假設process context與本CPU的interrupt handler都會取得spinlock L。process持L時若未禁止該interrupt，handler可搶入並spin等待L；但持有者被handler打斷而無法釋放，形成同CPU自我死鎖。對這種共享關係須使用相應irq-save鎖定協定，離場恢復原IRQ狀態，不能一律無條件開interrupt。

機制	能阻止	仍不能阻止
禁止本CPU preemption	scheduler切換本CPU的task	硬體IRQ、其他CPU並行
禁止本CPU普通IRQ	對應本CPU IRQ handler搶入	其他CPU、NMI
atomic spinlock	遵守同鎖協定的其他CPU/contexts	未用同鎖者、錯誤DMA與不合規存取
irq-save spinlock	同鎖SMP互斥及指定本地IRQ重入	所有未納入協定的其他機制

不能睡眠的critical path不應取得可能阻塞的mutex；spinlock持有區應短，否則浪費其他CPU。NMI與不同softirq/interrupt配置須另作協定分析，不能把教材的優先層圖當作涵蓋所有kernel configuration的總排序。

固定x86-32配置後，zones才有具體位址

本例固定Linux3.2的x86-32、4KiB pages、3GiB user/1GiB kernel split，正常direct mapping覆蓋前896MiB physical RAM，示範機有1GiB RAM。這是此配置，不能搬到所有32/64-bit Linux。zones表示位址coverage，不表示所有頁都free；kernel與reserved pages仍占用其中空間。

physical interval	zone	用途/限制
[0,16MiB)	DMA	支援特定legacy ISA DMA位址限制
[16,896MiB)	NORMAL	一般可由kernel direct map存取的RAM
[896MiB,1GiB)	HIGHMEM	不在永久direct map，kernel需適當暫時映射

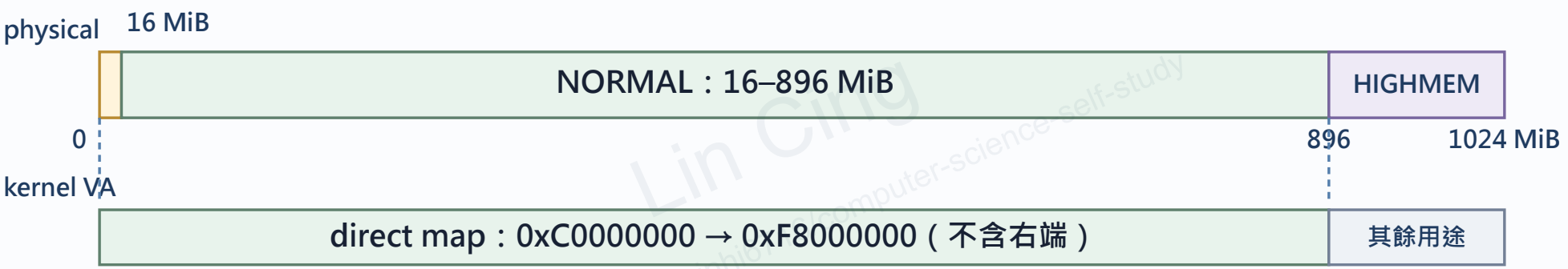
normal physical `0x01002000`可在這個模型以PAGE_OFFSET `0xC0000000`相加，得kernel VA `0xC1002000`。highmem physical `0x39000000`不能因此逕稱kernel VA=`0xF9000000`：那段kernel VA有其他用途，須先建立合法映射。user PTE仍可映射highmem frame；例如VA頁0x40000映至PFN0x39000，存取VA0x40000123得到PA0x39000123。

固定x86-32配置後，zones才有具體位址 （續）

physical page allocator依zone/order約束配置；buddy處理連續power-of-two pages，slab再管理小物件。
裝置能用的DMA位址還取決於device mask、IOMMU/bounce與DMA API，不是所有DMA都只能用前16MiB。

x86-32固定配置：physical RAM的zones與kernel direct map

1GiB RAM、3GiB/1GiB split；橫軸每1px代表1MiB，區間左閉右開。



黃：DMA [0,16)。綠：direct-mapped RAM。紫：須另建kernel mapping；不是直接加PAGE_OFFSET。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ELF的file bytes與zero-fill不能混算

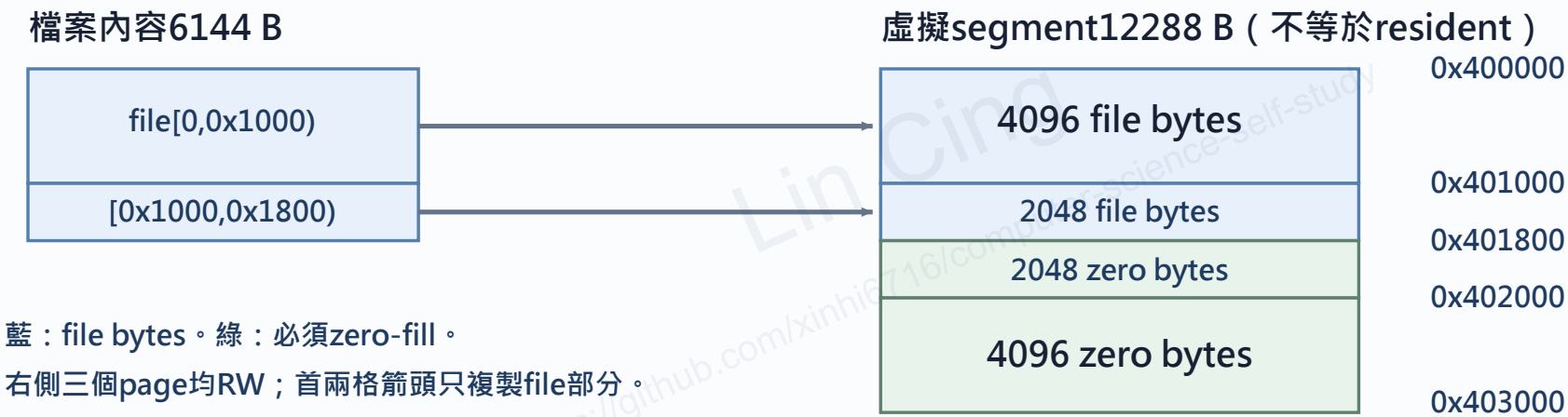
本段另採具NX支援的x86-64 ELF權限模型；地址只是例題指定值，與前頁x86-32 zones分開。固定一個PT_LOAD segment：p_offset=0、p_vaddr=0x400000、p_filesz=0x1800、p_memsz=0x3000、權限RW。p_memsz大於p_filesz的尾部必須讀回零，而非後續檔案垃圾。

virtual page	bytes來源	權限
[0x400000,0x401000)	file [0,0x1000)	RW
[0x401000,0x402000)	file [0x1000,0x1800)，再zero 0x800 bytes	RW
[0x402000,0x403000)	zero-fill 0x1000 bytes	RW

合計file-backed內容6144bytes、zero尾部6144bytes、virtual segment12288bytes；建立這個VMA不表示三頁都已resident。read-only code可由另一個RX PT_LOAD描述，private writes再依COW處理；由此連接18.4的fork與第9章fault path。

ELF的file bytes與zero-fill不能混算 （續）

動態ELF的PT_INTERP指定user-mode interpreter。kernel建立program/interpreter mappings與初始stack並交出入口，動態linker再解析DT_NEEDED與symbols。不能把所有動態連結說成kernel為每次函式呼叫找symbol，也不能要求所有共享libraries永遠映到同一個VA。



Page reclaim：保留後備資料與存活引用

回收時須分開追蹤映射、backing、dirty 狀態與是否可回收。若 CPU 或裝置仍有需要該 frame 的有效引用，就不能重用。乾淨的 file-cache page 只有在後備資料仍有效時才能重建；dirty page 可能是最新內容的唯一副本，必須完成所需 writeback，才能將其涵蓋的狀態標成 clean。

回收協定還須移除不安全的映射與使用者，並釋放必要引用；應用程式暫時沒有執行，不代表頁面不再被引用。writeback 完成也不自動等於整個檔案系統交易已持久化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VFS四種物件接上buffered read

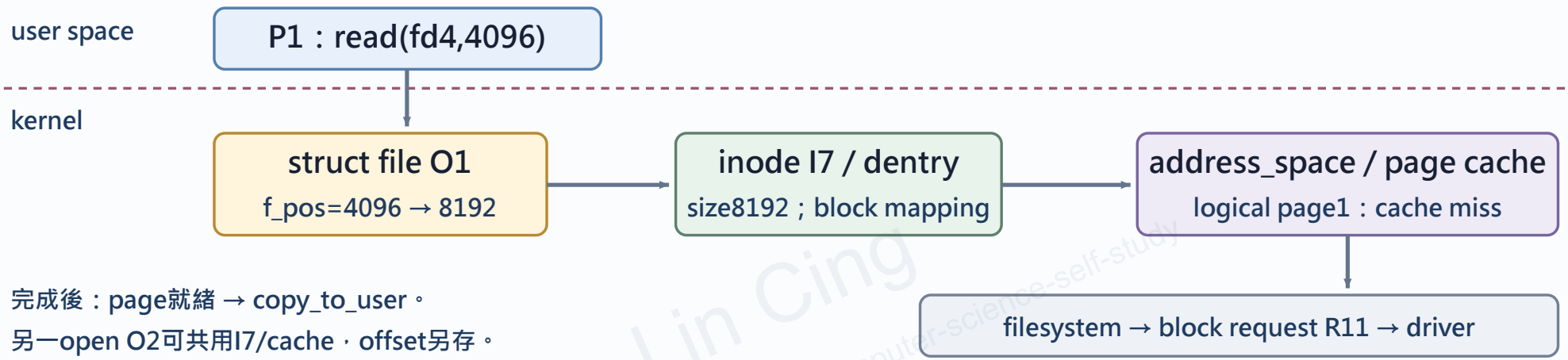
固定Linux3.2、一般ext3 buffered read、file size8192、page4KiB、P1 fd4已成功open；fd4指struct file O1，O1.f_pos=4096，引用檔案dentry與inode I7。I7的address_space管理共享file pages，superblock表示所屬filesystem。本例另假定讀取完整成功、資料可用、copy_to_user 全部成功，且沒有並行 truncate 或共享 offset 的變更。

物件	保存/連結	不應混入
superblock	filesystem instance與operations	每次open的offset
dentry	parent/name與inode關係，可含negative lookup	檔案的唯一名字假設
inode	身分、mode、size、block mapping、address_space	所有open共用的current offset
struct file O1	本次open的f_pos、flags、operations與path引用	每個fd都必然獨立offset的假設

P1 read4096先以fd取得O1引用，驗證操作與user buffer，再走file operations；若logical page1不在page cache，filesystem把它的blocks送到I/O，等待page就緒後copy到user，O1.f_pos=8192。另一個獨立open有O2，但可共用I7的同一page cache；dup(fd4)則共用O1。cache hit可省block I/O，不省所有查找/copy成本。

VFS四種物件接上buffered read （續）

buffered read：open file的offset與共享inode/page cache分開



完成後：page就緒 → copy_to_user。

另一open O2可共用I7/cache，offset另存。

虛線：user/kernel界線。箭頭：物件引用或此read的呼叫路徑；冷cache才需下送R11。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ext3、block groups與procfs的語意邊界

ext3以inode、data blocks、bitmaps與block groups配置資料，嘗試維持locality；鄰近配置是政策目標，不能當作「同一file永遠連續」的正確性不變量。檔案增長或碎片可迫使新block落到其他位置，索引仍須正確。

固定 `data=ordered` 的教材模型：write 6000 bytes 使兩個 page-cache pages 變 dirty，且此次 metadata transaction 要公開新配置的 block mapping。相關資料在公開該 mapping 的 metadata commit 前須按此模式的排序要求處理，以避免新 metadata 指向未就緒內容。普通write成功不等於fsync成功，journal的metadata原子性也不保證所有in-place user data修改都原子。`data=writeback`或`data=journal`是不同模式，不能共用同一句持久承諾。

procfs的`/proc/123/status`在read時由kernel狀態形成文字，不一定對應一塊持久disk data。PID123會在process結束後被重用，因此稍後同一路徑未必是同一個process；系統監測需結合啟動時間或其他身分證據。多個proc檔案分開讀取也不自動形成全系統同一時刻的snapshot。本章不採「PID固定16bit、inode高16bit永遠編碼PID」作Linux3.2通用規格。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

block、character與network三條I/O路徑

block device提供依block/sector定位的資料；filesystem透過block mapping及request layer送到driver。character device以該driver的file operations定義串流或其他操作，不保證都可seek。network device經socket/protocol stack送packet，不等同把所有網卡都當作可任意seek的device file。

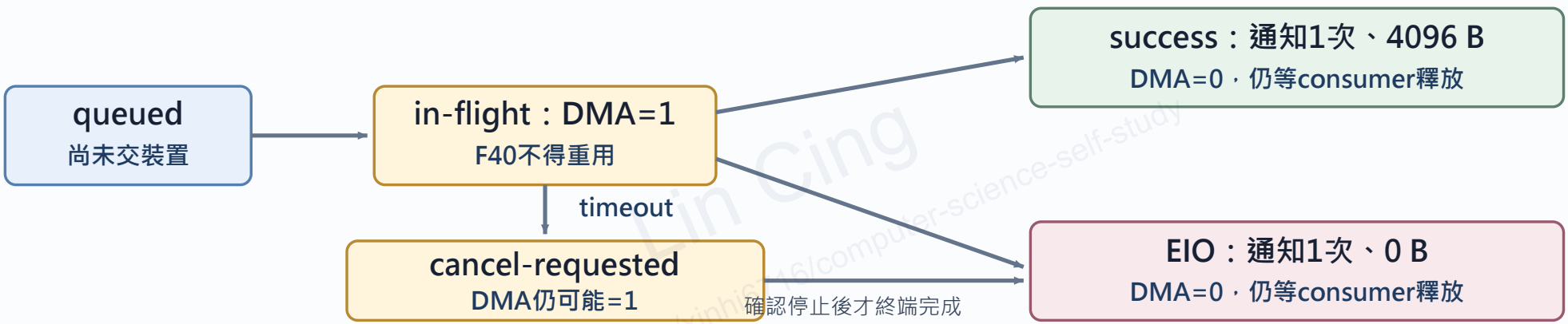
固定同步read一頁、cache cold、user buffer已resident、無read-ahead，request R11指向一頁kernel buffer F40。state依序queued→in-flight(DMA持有F40)→terminal success或terminal error；完成後才能發布結果並按生命週期解除mapping。實際block layer可merge/split requests，本例一頁一request只是一個固定輸入。

結局	device/DMA證據	終端callback	app結果	F40何時可重用
success	I/O完成、ownership歸還	恰1次	本例讀4096bytes	copy/引用結束後
error EIO	本request已停止或失敗完成	仍恰1次	沒有先前成功bytes時回錯誤	DMA已停且引用結束後
timeout但裝置仍可能工作	尚未證明停止	不能先當成功/安全釋放	等取消/重置協定	尚不可

block、character與network三條I/O路徑 (續)

錯誤完成也是完成，不能把EIO當作「completion次數不增加」。反過來，timeout不是裝置已停止的證明；若過早free F40，遲到的DMA會改壞新owner資料。

R11、generation3：timeout不能直接當成DMA已停止



黃：仍可能使用buffer。綠/紅：成功/錯誤完成。相同或舊generation的重複通知不再free一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

duplicate completion與reset的狀態機

本例以`(request id=11,generation=3)`識別一次提交。第一個合法terminal事件以受保護狀態把in-flight→terminal並通知一次；再來同generation的重複事件只記錄/忽略，不再通知或free一次。下一次重用id11時generation=4，舊generation3事件不能完成新request。

這個generation只處理**軟體事件身分**，不是停止硬體DMA的替代品。reset/cancel路徑還須確認裝置不再使用舊DMA address，完成相應同步後才能解除mapping、回收buffer。driver需按裝置協定處理late interrupt與error，不可只靠一個timeout數字判定安全。

user buffer可被釋放的時點也依同步/非同步API不同：同步read返回前copy已完成；非同步提交後則要等completion契約。request queue、memory ownership與通知次數須連在同一張狀態表上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

pipe的EOF由所有writer引用決定

固定pipe含read description R、write description W；parent fd3→R、fd4→W，fork複製descriptor table後，child fd3/fd4仍引用同一R/W objects。parent將讀交給consumer、child負責producer時，必須關閉各自不用的端點。

事件	parent writer	child writer	buffered bytes	read端結果
fork後	1	1	0	阻塞或nonblocking EAGAIN，並非EOF
child寫ABC、關fd4	1	0	3	可讀ABC
consumer讀完	1	0	0	仍未EOF，因parent誤留writer
parent關最後writer	0	0	0	後續read回0即EOF

同一stream保留bytes次序，但多個writers的操作原子性受PIPE_BUF與write大小/模式限制，不能從「是pipe」推出任意長write永不交錯。shared memory只共用資料，不提供queue/互斥；需另選locks、semaphores或atomic protocol。

signals、wait queues與原子semop

standard signals在blocked期間同一signal多次產生可合併成一個pending事件；不能用它當可靠計數器。real-time signals可排隊，sigqueue等介面可帶受限payload，故不能說「所有signals都不能傳資料」。處理函式可用的操作還受async-signal-safety限制。

kernel內等待I/O常用condition加wait queue。若consumer先查done=false，producer立刻設done=true並wake，consumer卻之後才入queue並sleep，就可能漏喚醒。正確等待協定先建立可被喚醒的等待狀態，再重查condition，若已true就不sleep；被喚醒後仍重查，因其他waiter可能先消耗資源。wake把task變runnable，不保證它立刻執行或condition永久成立。

System V semop可把同組多個semaphore操作合成原子批次。初值S0=1,S1=0，請求`S0-=1;S1-=1`在兩者都足夠前不部分扣S0；若先把S0扣成0再等待S1，就不是此原子批次語意。IPC權限、資源上限、刪除與錯誤返回仍需明定，不能只列介面名稱。

從socket到wire，資料量的分母各不同

固定IPv4/TCP、無options、MTU1500、無TSO/GSO等offload；application成功send6000bytes。MTU扣除標頭後容許最多1460 B，但實際分段還受對端MSS、路徑MTU、排程及窗口等影響。本例另指定有效MSS恰為1460 B、無重傳且先送四個滿載段與一個尾段，wire上的五個TCP payloads為1460、1460、1460、1460、160，合計6000；五組IP/TCP headers共200bytes，L3 transmitted bytes=6200，尚未計Ethernet overhead、ACK或重傳。

kernel socket buffer/sk_buff、TCP send queue、IP routing與network driver各保存不同狀態。若啟用segmentation offload，一個較大sk_buff仍可在硬體分成多個wire packets，因此sk_buff數不能直接當wire packet數。send成功只承諾API已接受的bytes，不等於remote app已讀、對方已ACK，更不等於磁碟持久。

routing例子：目的10.1.2.3，同時存在default /0、10.0.0.0/8、10.1.0.0/16及10.1.2.0/24，最長匹配是/24。FIB實際資料結構、route cache與firewall hooks依版本/CONFIG；本章保留可核對的longest-prefix語意，不把舊書某一hash/cache布局宣稱為所有Linux版本。

queue、retransmission與延遲的證據

固定一筆量測trace，以enqueue時刻0為基準：dispatch=2ms、device transmit完成=3ms、所需ACK到達=13ms。因此queue等待2ms、已派送到transmit完成1ms、其後到ACK10ms，這是分段timestamp支持的結論。

只有queue length20、retransmissions3、device errors1三個聚合計數，不能推出「長尾就是queue造成」；重傳可能來自網路丟失、重排或其他原因，device error也未必與該TCP流一致。需要同一request/flow、同一時間窗的enqueue/dispatch/completion、drop與ACK evidence，或明確干預比較，才能分辨因果。

TCP按sequence numbers處理重傳與out-of-order bytes，供應成功交付的有序byte stream；UDP保留datagram界線但不保證交付。IP/driver queue本身不因上層TCP就成為不會丟失或重排的通道。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Authentication與authorization各回答一個問題

authentication確認宣稱的身分；authorization判斷該credentials對特定object/operation有何權限。PAM把authentication/account/session等政策交給user-space模組組合，kernel仍依其credentials執行檔案與其他資源的檢查。密碼salt讓相同密碼不必有相同儲存值，但不增加密碼本身entropy；hash的成本參數與存取保護是另一部分，本章不把舊UNIX八字元機制當現行建議。

Linux credentials包含real/effective/saved IDs、filesystem IDs、groups與capabilities等。對檔案的普通DAC，先選owner/group/other類別，再看requested read/write/execute；capability、ACL/LSM、mount條件可進一步改變結果。成功open還可能形成持續可用的handle，日後只改path mode並不等於撤銷所有既有handle。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

root並不等於「mode沒有write就一定被拒絕」

固定Linuxv3.2、使用 generic_permission 的非 immutable regular file，owner1000、mode0400、rw mount、無ACL及其他LSM拒絕；路徑 traversal 已成功，caller current_user_ns 與 inode_userns 相同。每次獨立要求單一 read、write 或 execute 權限。下表capability指在對應user namespace中的有效DAC能力，忽略其他額外政策。

caller	read	write	execute	規則
fsuid1000，無DAC能力	允許	拒絕	拒絕	owner bits=100
fsuid2000，無DAC能力	拒絕	拒絕	拒絕	other bits=000
fsuid0，有CAP_DAC_OVERRIDE	允許	允許	拒絕	能越過read/write DAC；regular file仍須至少一個 execute bit
fsuid0，但無相關DAC能力	拒絕	拒絕	拒絕	UID數字本身不補上缺少的能力
fsuid2000，只有 CAP_DAC_READ_SEARCH	允許	拒絕	拒絕	regular file只補read，不補write/execute

root並不等於「mode沒有write就一定被拒絕」 (續)

v3.2 `generic_permission()`先做一般檢查，失敗後對regular file檢查CAP_DAC_OVERRIDE；涉及execute時還要求`S_IXUGO`非零。若mode改0401，有DAC_OVERRIDE者的execute DAC可通過；是否可真正執行仍受格式、mount noexec與其他政策限制。這直接反駁「root write因mode未開而必deny」，也避免把root說成無條件穿過所有安全層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

暫時降權與永久降權的差別

固定setuid-root程式的IDs為real1000、effective0、saved0。`seteuid(1000)`後real仍1000、effective1000、saved0；之後可依saved-ID規則恢復effective0。這是暫時放棄有效UID，不是永久移除全部特權。

若要永久降權，還須按相應API處理saved IDs、groups與capability sets，並關閉不應保留的privileged handles；僅改euid不能撤銷早已開啟的file descriptors。具體實作還受securebits、user namespaces與程式所需資源影響，所以本章表格只證明ID轉移，不把一段省略其他credentials的偽碼當成完整sandbox。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把read的一次成功與一次失敗接回同一機制

固定18.7的fd4→O1→I7、offset4096、read4096、page cache cold、user buffer已resident；沒有並行 truncate 或共享 offset 更新，I/O 和 copy_to_user 均完整成功。正向鏈為fd引用/操作檢查→page cache查找→filesystem mapping→R11 block I/O→device完成→page就緒→copy_to_user→O1.offset8192→回傳4096。等待中task可被切換；這與函式呼叫不是同一事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

錯誤完成、部分進度與物件生命週期

若R11以EIO終端完成且尚未copy任何bytes，read回錯誤，offset不因此前進；完成通知仍恰一次，buffer必須等DMA停止與references釋放才重用。若已有部分copy，API可能回傳成功bytes再於後續觀察錯誤，不能把所有read失敗都硬改成零進度。本例排除了先前成功bytes，才有確定的零進度結果。

fork共享資料頁不代表共享task，兩次open共享inode不代表共享offset，send接受bytes不代表遠端接收，write修改cache不代表持久；這些界線由物件身分、權限、生命週期與完成契約共同維持。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study